Mobile phone Bluetooth open door SDK development guide

IOS

V2.4.3

IOS SDK Version release record

Release version	date of issue	Update content	developer
V1.4 (beta version)	2016.02.02	1. SDK multi-user setting and switching. 2. Optimization of electronic key door opening interface.	Jason.Huang
V1.5	2016.03.29	1. Add the SDK support running environment: i386, x86_64, arm64, armv7, armv7s. 2. Repair V1.4 version bug: Framework SDK for dynamic library does not support shelf, switch to static library compilation. 3. Optimize Bluetooth door opening: Connect the Bluetooth device failed to reconnect 3 times.	Jason.Huang
V1.6	2016.04.26	1. Add the open-door extension interface, openDoor. 2. Open opening operation adds shelf life verification. 3. Remove the SDK data function, namely the setDevice and setUser interfaces, and the data is stored by the caller app; delete reason: the storage used by SDK conflicts with the app.	Jason.Huang
V1.7	2016.07.11	1. Optimize and improve the scanning opening time, and improve the accuracy of the internal scanning multiple value. 2. Add the function of card registration, card deletion, synchronization of device time, and setting reading format.	Jason.Huang
V1.8	2016.10.11	1. Add close to the door opening function interface 2. Add the device parameter setting interface 3. Increase initialization preloaded Bluetooth, do not open Bluetooth prompt box function initBluebоothNotShowPower	Jason.Huang
V1.9	2016.12.30	1. Optimize close door performance 2. Repair the ladder-controlled door-opening bug	Jason.Huang
V2.0	2017.03.20	1. Add the modified device password interface 2. Optimize the interface parameters of the scanning equipment: the scanning time unit	Jason.Huang
V2.0.1	2017.04.11	1.Modify the setDeviceParam method named as: setDeviceConfig 2.Add the interface to the acquiring equipment system parameters	Benson.chen
V2.1.0	2017.08.03	1. Add the bright screen door opening interface	Benson.chen
V2.1.1	2017.08.29	1. Remove the monitoring of lock screen	Benson.chen

		notifications to avoid Apple review	
V2.1.2	2017.09.20	1. Add the batch card issuing interface, configure the card reading sector interface, and configure the WiFi interface	Benson.chen
V2.1.3	2017.10.23	1. Increase the EXT200, EXT210, and EXT211 Bluetooth transmission card number	Benson.chen
V2.1.4	2018.01.15	1. Increase the stability of Bluetooth communication 2. Add the function of synchronous fingerprint data 3. Add the function of obtaining the card number 4. Add the access door record function 5. Optimize the shelf-life validation	Benson.chen
V2.1.5	2018.03.14	1. Obtain equipment information and increase how to obtain power information	Benson.chen
V2.1.6	2018.03.24	1. Add configuration device IP interface and connection server IP interface (for V500 series)	Benson.chen
V2.1.7	2018.04.27	1. Optimize the shelf-life validation 2. Remove the bright screen door opening interface	Benson.chen
V2.1.8	2018.05.07	1. Repair a bug with incorrect MAC address resolution	Benson.chen
V2.1.9	2018.05.16	1. Repair the bug where the automatic opening interval does not take effect	Benson.chen
V2.2.0	2018.05.21	1. Repair the bugs that are hung by the system during a long automatic door opening interval	Benson.chen
V2.2.1	2018.06.01	1. Repair a bug that cannot be closed multiple times	Benson.chen
V2.2.2	2018.06.23	1. Increase access to unread door opening records 2. Add the deleted door opening records	Benson.chen
V2.2.3	2019.03.27	1. Add three types of SL100B, SL200B and SL300B (device password in cardno)	Benson.chen
V2.2.4	2019.04.01	1. Fix the write card bug 2. Add the delete card interface	Benson.chen
V2.2.5	2019.04.27	1. Repair the scan added	Benson.chen
V2.2.6	2019.10.21	1. Improve the batch write card function	Benson.chen
V2.2.7	2019.11.14	1. Increase the SL400B, SL500B type	Benson.chen
V2.2.8	2019.12.28	1. Repair of the known bugs	Benson.chen
V2.2.9	2020.04.21	1. Repair gets the nearest record bug: timeout with no callback	Benson.chen
V2.2.10	2020.08.26	1. Increase the card number to get the card added	Benson.chen
V2.2.11	2020.08.27	1. Automatic upload of the door opening record (for the new platform)	Benson.chen
V2.2.12	2020.09.05	1. Add an interface to acquire the device signal	Benson.chen

V2.3.0	2020.10.17	1. Optimize the protocol	Benson.chen
V2.3.3	2021.01.13	1. optimize	Benson.chen
V2.3.4	2021.02.04	1. Add a one-button door opening interface	Benson.chen
V2.3.5	2021.02.05	1. One key to open the door of a new intermediate process callback	Benson.chen
V2.3.6	2021.02.08	1. Improve access to Sim card information and add the module name	Benson.chen
V2.3.7	2021.03.04	1. New device registration fingerprint 2. New get all fingerprint id 3. New setting of APN 4. Get APN 5. Add to set the NB lock networking parameters 6. Add the set NB lock network password	Benson.chen
V2.4.0	2022.01.25	1. Add the function of frequency clearing (nb) 2. New access NB base station information 3. Add new Settings and obtain China Unicom nb platform parameters 4. New cpu card key configuration and read 5. New configuration and read of the read card type card authentication type 6. Whether the new setting anti-replication function is enabled 7. New weigen parameter configuration 8. Add a regular open and regular close setting	Benson.chen
V2.4.2	2022.06.21	1. Repair the empty card exception 2. New configuration and read access control mode function	Benson.chen
V2.4.3	2023.03.21	1. Add WiFi interface for host device 2. Added upgrade enabling interface 3. New read host network status interface 4. Add the reading host device information interface 5. Add the set host apn interface 6. New read host apn interface 7. Add the set host communication type interface 8. Add an interface to set whether the host is often open 9. Add the interface to read whether the host is often open	Benson.chen

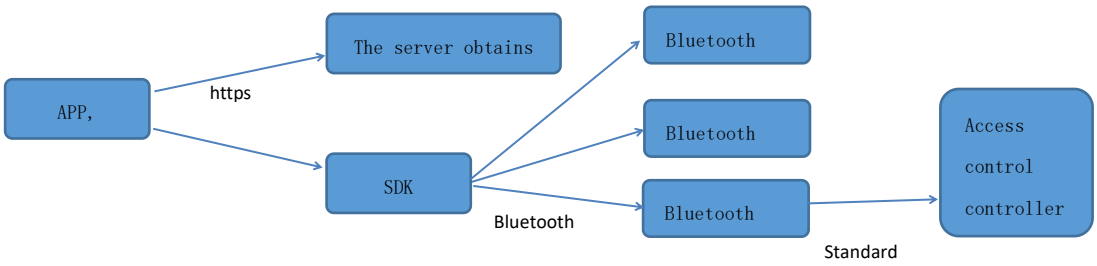
Document purpose

Detailed the development of app mobile terminal and the interface and precautions provided by SDK. This document is developed as an internal app and provided for third-party app development.

Read before development

Bluetooth sdk provides the login method. After calling the login method, the result is automatically reported to the server after opening the Bluetooth door. See 4.1 for the Bluetooth login method

1. APP mobile terminal, SDK, server open platform, equipment, communication process



2. Functional description

The APP mobile terminal currently supports Android mobile phone and iphone mobile phone. The mobile phone door opening function actually uses the electronic key. The source of the electronic key is requested from the cloud server. After obtaining the electronic key allocated by the cloud, the APP can realize the mobile phone door opening function by calling the SDK interface to operate the specified device.

Mobile terminal equipment	System version	Bluetooth version
Android Mobile phones	Android 4.3 and above	4.0, and above
iPhone Mobile phones	ios 7.0 and above	4.0, and above

3. Notes for SDK use

Mobile phone Bluetooth open the door IOS SDK, using the static library Framework, static library name: DoorMasterSDK.framework, refer to the demo example. For the detailed use mode of the static library, please refer to the following website:

Static library: http: / / www.cocoachina.com/ios/20150127/11022.html

For static library calls, refer to the following website:

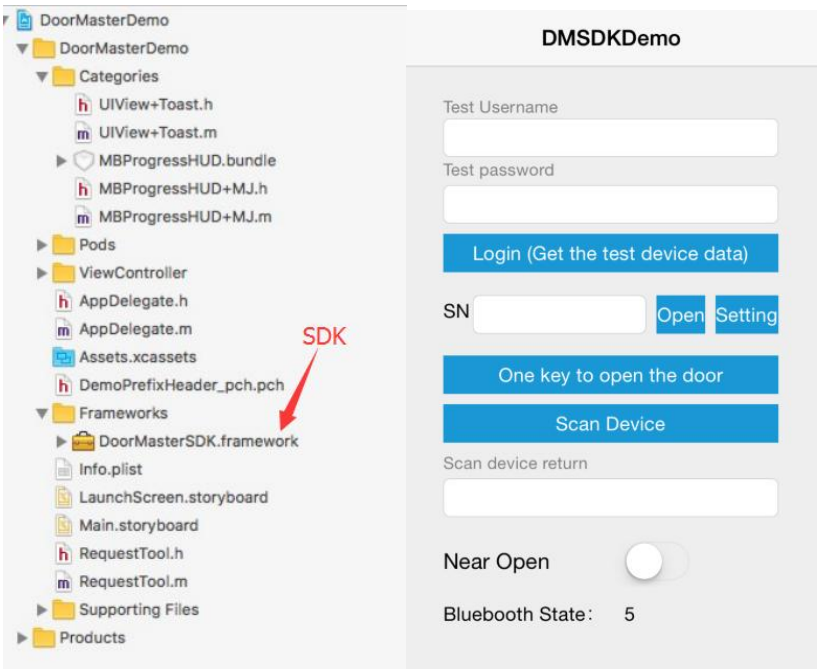
http://blog.csdn.net/hengshujiyi/article/details/21182813

The following two pictures are Demo structure: 1. Use the registered APP account to log in to obtain the user's authorized device (contact us to obtain the test account), and open the device by clicking the door button nearby. If there are multiple devices to test, please enter and select the device serial number by yourself.

The Demo usage steps:

1. If you need to test the door, you need the secret key of the device, you can log in the cloud server to obtain the authorized device secret key. As long as you have the secret key of the device, the door can be realized by calling the door interface, and the secret key of the device can also be generated by your own server.

2. Close to the door opening function requires equipment to test.



4. The SDK interface function

4.1 login

[function]

Log in to the server and automatically upload the door opening record after Bluetooth door opening.

The login function is the static method of the LibDevModel class

[function]

+(int)login :(NSString *)domian appId:(NSString *)appId appSecret:(NSString *)appSecret account:(NSString *)account callback :(BlockOnControlOver)callback ;

[parameter declaration]

domain: The server address of the login, the default transmission https: / / app-API.thinmoo.com

AppId: login appId

appSecret: login appSecret

account:, Login app account

callback: A callback.

[App call SDK Example]:

[LibDevModel login:@” https://app-api.thinmoo.com ” appId:appId appSecret:appSecret account:account callback:^(int ret, NSMutableDictionary){

```
        / / ret=0 login is successful
    }];
```

4.2 initBluetooth and initBluetoothNotShowPower

[function]

Initialize Bluetooth. It takes time to open Bluetooth, so you need to initialize to improve the speed. It is recommended that the App call the interface to initialize Bluetooth. See Demo code for detailed use.

The initBluetooth and initBluetoothNotShowPower functions are static methods for the Lib UserModel class, Difference: When the mobile phone Bluetooth is not open, the initBluetoothNotShowPower does not pop up the Bluetooth prompt box.

The return values are detailed in Schedule III.

[function]

```
+(int) initBluetooth;
+(int) initBluetoothNotShowPower;
```

4.3 onBluetoothStateOver

[function]

initBluetooth After initializing Bluetooth, the interface can be called to delegate monitor the Bluetooth state, and the Bluetooth state change is returned immediately. See Demo code for details.

The onBluetoothStateOver function is the static method of the Lib UserModel class.

The return values are detailed in Schedule III.

[function]

```
+(void ) onBluetoothStateOver:(BlockBluetoothStateOver)blockBluetoothState;
```

[parameter declaration]

blockBluetoothState: BlockBluetoothStateOver returns the value type definition and returns the status value

[returned value]

Return the status value	state description
0	CBCentralManagerStateUnknown
1	CBCentralManagerStateResetting
2	CBCentralManagerStateUnsupported
3	CBCentralManagerStateUnauthorized
4	CBCentralManagerStatePoweredOff
5	CBCentralManagerStatePoweredOn

4.4 controlDevice

[function]

Control device door opening and restart operations. See Schedule II for more features.

The controlDevice function is a static method of the LibDevModel class and needs to call the onControlOver callback function to get the operation result.

The return values are detailed in Schedule III.

[function]

+(int)control Device:(LibDevModel *)devModel andOperation:(int)operation;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

operation: For details, please refer to Attached Table II.

[App call SDK Example]:

```
LibDevModel *devModel = [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 1; / / Device type
devModel.cardno = @ "* * * * * * * * * *"; / / Card number
devModel.startDate = @ ""; / / permanent
devModel.endDate = @ ""; / / permanent
int ret = [LibDevModel controlDevice:devModel andOperation:0];
[LibDevModel onControlOver:^(int ret, NSMutableDictionary *msgDict) {
    // code
}];
```

4.5 onControlOver

[function]

Operating the callback function of the device to get the result of the operation.

The onControlOver function is the static method of the LibDevModel class.

The return values are detailed in Schedule III.

[function]

+(void)onControlOver:(BlockOnControlOver)blockControl;

[parameter declaration]

blockControl: The operating device returns the results.

BlockOnControlOver Type Definition: typedef void (^BlockOnControlOver) (int ret, NSMutableDictionary * msgDict).

4.6 openDoor

[function]

Lock the control device

The openDoor function is a static method of the LibDevModel class and needs to call the onControlOver callback function to get the operation result. The return values are detailed in Schedule III.

[function]

+(int)openDoor :(LibDevModel *)devModel;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

[App call SDK Example]:

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device Serial Number (required)
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC (required)
devModel.eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key (required)
devModel.devType = 1; / / Device type (required)
devModel.cardno = @ "* * * * * * * * *"; / / card number (must pass, the user has the real card number, pass the real card number, otherwise pass "0000000000")
devModel.startDate = @ ""; / / Permanent (mandatory)
devModel.endDate = @ ""; / / Permanent (mandatory)
int ret = [LibDevModel openDoor :devModel ];
[LibDevModel onControlOver:^(int ret, NSMutableDictionary *msgDict) {
    // code
}];
```

4.7 s canDevice

[function]

Scan nearby Bluetooth reading head, all-in-one, etc., can be used for app device list online device highlight.

scanDevice Function is a static method of Lib DevModel class. After scanning, onScanOver

callback function to get the scan result.

The return values are detailed in Schedule III.

[function]

+(int) scanDevice:(int)scanTime;

[parameter declaration]

s canTime: Scan time, in milliseconds.100 – 600 ms indicates scanning door opening and 1000 – 10000 indicates scans only the device.

[APP call example]

```
@implementation ViewController
int ret = [LibDevModel scanDevice: 100]; / / Scan the door
/ / int ret = [LibDevModel scanDevice: 1000]; / / Scan the add device
[LibDevModel onScanOver:^(NSMutableDictionary *devDict) {
      // operation, see demo code
}];
@end
```

4.8 onScanOver Sort

[function]

Scan the device's callback function to get the results of the scan.

The onScanOver Sort function is the static method of the LibDevModel class.

The return values are detailed in Schedule III.

[function]

+(void)onScanOver Sort :(BlockOnScanOver Sort)blockScan;

[parameter declaration]

blockScan: The scanning device returns the results.

BlockOnScanOver Sort Type definition: typedef void (^BlockOnScanOver) (NS Array * dev List);

dev List format: [{device serial number: signal value}], is an orderly array, according to the strength of the signal value, the stronger the signal, the array elements is the dictionary, the dictionary only a pair of key and value, key is the serial number, value is the signal value. Example: [{"123456": -70}, {"654321": -82}]

4.9 syncDeviceTime

[function]

Synchronize equipment time, used for temporary password door opening, supported

equipment: access control all-in-one machine, wireless lock equipment, the equipment must have a key function.

The syncDeviceTime function is a static method of the Lib DevModel class, and you need to call the onControlOver callback function to get the operation result.

The return values are detailed in Schedule III.

[function]

+(int)syncDeviceTime:(LibDevModel *)devModel andTime:(NSString *) time;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

Time: synchronization, format: 15110910042301, meaning: November 09,10:04:23-Monday, can also be imported 201511 0 910042301.

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 1; / / Device type
int ret = [LibDevModel syncDeviceTime :devModel andTime:@”2015110910042301”];
[LibDevModel onControlOver:^(int ret, NSMutableDictionary *msgDict) {
    // code
}];
```

4.10 modify Pwd

[function]

Change the equipment management / door password, support equipment: access control all-in-one machine, wireless lock equipment, the equipment must have key function.

modify Pwd, Function is a static method of Lib DevModel class, you need to call the onControlOver callback function to get the operation results.

The return values are detailed in Schedule III.

[function]

+(int)modify Pwd :(LibDevModel *)devModel and OldPwd :(NSString *) oldPwd andNe wPwd:(NSString *)newPwd ;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

oldPwd: Old password, 6 digits; no initial password by default, enter @ ".

newPwd: New password, 6 digits.

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 1; / / Device type
int ret = [LibDevModel modify Pwd :devModel and OldPwd :@”” andNewPwd:@”123456”];
[LibDevModel onControlOver:^(int ret, NSMutableDictionary *msgDict) {
    // code
}];
```

4.11 s etDevice Config

[function]

Synchronize equipment parameter information to set weigen format, opening time and electrical switch signal setting.

The s etDevice Config function is a static method of the Lib DevModel class and needs to call the onControlOver callback function to get the operation result.

The return values are detailed in Schedule III.

[function]

+(int)s etDevice Config :(LibDevModel *)devModel andWGFmt :(int)wgfmt andDriverTime:(int)driverTime andRelaySwitch:(int)relaySwitch ;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

wgfmt: ergen format, currently only support 26,34.

driverTime: Door lock open time, default 5s. Value range: 1-254.

relaySwitch: Electrical switch signal setting (0 electrical lock control, 1 electrical switch), default electrical lock control.

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 1; / / Device type
int ret = [LibDevModel s etDeviceParam :devModel andWGFmt :26 andDriverTime:5 andRelaySwitch:0];
```

```
[LibDevModel onControlOver:^(int ret, NSMutableDictionary *msgDict) {
    // code
}];
```

4.12 getDeviceConfig

[function]

Obtain the configuration of the current device (door opening time, card registration number, number of mobile phone registration, maximum user capacity, device control mode, weigen format, device version number).

The getDeviceConfig function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)getDeviceConfig:(LibDevModel *)devModel andGetDeviceConfigCallBack: (BlockOnControlOver) deviceConfigCallBack;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

deviceConfigCallBack: Type Definition: typedef void (^BlockOnControlOver) (int ret, NSMutableDictionary * msgDict).

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 1; / / Device type
int ret = [LibDevModel getDeviceConfig:libDevModel andGetDeviceConfigCallBack:^(int ret, NSMutableDictionary *msgDict) {
if (ret == SUCCESS)
{
[msgDict [@ "wgfmt"] intValue]; / / Get the Vergan parameter
[msgDict [@ "openTime"] intValue]; / / Get the opening time
[msgDict [@ "lockSwitch"] intValue]; / / Get locking parameters (0 electrical 1 electrical lock)
[msgDict[@"cardCount"] intValue];       / / Number of cards to be registered
[msgDict [@ "maxCardCount"] intValue]; / / Get the maximum card capacity
[msgDict [@ "userCount"] intValue]; / / Number of the users obtained
msgDict[@"version"];                    / / Get the firmware version number

[msgDict [@ "doorNo"] intValue]; / / door number
[msgDict [@ "section"] intValue]; / / sector
msgDict [@ "sectionKey"]; / / sector key
[msgDict [@ "devType"] intValue]; / / Device type
```

```
        msgDict [@ "serverIP"]; / / Connect to the serverIP
           msgDict [@ "serverPort"]; / / Connect to the server port
           msgDict [@ "WiFiName"]; / / WiFi name
           msgDict [@ "WiFiPwd"]; / / WiFi password
           msgDict [@ "date"]; / / Device time
           msgDict [@ "function"]; / / Device function parameters
           [msgDict [@ "electricity"] intValue]; / / Equipment charge: 1~100
}
else
{
/ / Obtain failure to view Schedule III according to the ret return value
}
}];
```

4.13 startBackgroundMode

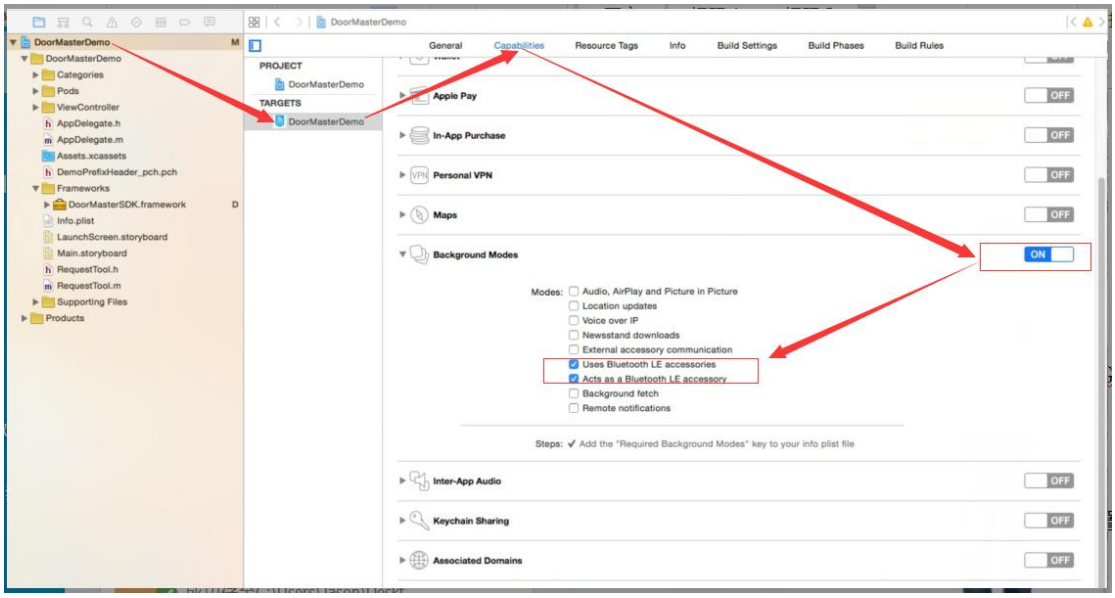
[function]

Start the SDK Bluetooth background mode, that is, to realize the function of close to the door.

The startBackgroundMode function is a static method of the Lib DevMode l class and needs to call the onBGScanOver callback function to get the operation result.

The return values are detailed in Schedule III.

Note: Use the Bluetooth background mode, as shown in the figure below:



[function]

+(int)startBackgroundMode ;

[APP call example]

```
[LibDevModel startBackgroundMode]; / / Start the SDK background mode
// Set the device callback function scanned back from the receiving background mode
[LibDevModel on BGScan Over:^(NSMutableDictionary *scanDev Dict) {
    // operation, see demo code
}];
```

4.14 stopBackgroundMode

[function]

Turn off the SDK Bluetooth background mode, that is, turn off the close door function.

stopBackgroundMode Function is a static method of class Lib DevModel. See Schedule 3 for details of the return value.

[function]

```
+(int)stopBackgroundMode ;
```

[APP call example]

```
[LibDevModel stopBackgroundMode]; / / Close the close door function
```

4.15 configWiFi

[function]

Configure device connection routes that support WiFi functionality.

The configWiFi function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)configWiFi:(LibDevModel *)devModel ipAddress:(NSString *)ipAddress port:(int)port WiFiName:(NSString *)WiFiName WiFiPwd:(NSString *)WiFiPwd andCallBack:(BlockOnControlOver)callBack;
```

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

ipAddress: Server IP address.

Port: the server port.

WiFiName: Router name (SSID recommended in English)

WiFiPwd: Router password

callBack: Type Definition: typedef void (^BlockOnControlOver) (int ret, NS MutableDictionary * msgDict).

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
```

devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number

devModel.devMac = @ "* * * * * * * * * * * * *"; / / Device MAC

devModel. E eKey = @ "* *"; / / User electronic key

devModel.devType = 13; / / Device type

int ret = [LibDevModel configWiFi:devModel ipAddress:@”120.24.1.12” port:8181 WiFiName:@”fjklasfdsil ” WiFiPwd:@”123456789” andCallBack :^(int ret, NSMutableDictionary *msgDict) {

```
if (ret == SUCCESS)
{
/ / Configuration was successful
}
else
{
/ / Configuration failure View Supplementary Table III as per the ret return value
}
}];
if(ret != 0)
   {
        / / Configuration failed, view attached Table III from the ret return value
   }
```

4.16 setReadSectorKey

[function]

Configure the device number, read the card sector number, card sector key, for the M160 access control machine and the temporary password configuration of all all-in-one machines.

The setReadSectorKey function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)setReadSectorKey:(LibDevModel *)devModel devId:(int)devId mifareSector:(int)mifareSector sectorKey:(NSString *)sectorKey andCallBack:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

DevId: Device id number (0~255)

mifareSector: Mifare sector address (0~15)

sectorKey: sector key (hexadecimal string, length 12)

callBack: Type Definition: typedef void (^BlockOnControlOver) (int ret, NS MutableDictionary * msgDict).

[APP call example]

LibDevModel *devModel= [[LibDevModel alloc] init];

devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number

devModel.devMac = @ "* * * * * * * * * * * * *"; / / Device MAC

devModel. E eKey = @ "* *"; / / User electronic key

devModel.devType = 13; / / Device type

int ret = [LibDevModel setReadSectorKey:devModel devId:5 mifareSector:1 sectorKey:@” 123456789abc ” andCallBack :^(int ret, NSMutableDictionary *msgDict) {

if (ret == SUCCESS)

{

/ / Configuration was successful

}

else

{

/ / Configuration failure View Supplementary Table III as per the ret return value

}

}];

if(ret != 0)

{

/ / Configuration failed, view attached Table III from the ret return value

}

4.17 writeCardToDevice

[function]

Write the card operation interface, and write the card number to the device in batch.

The writeCardToDevice function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)writeCardToDevice:(LibDevModel *)devModel andCards:(NSArray *)cardArray;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

cardArray: Card number data, card number is ten decimal, the number of card numbers written each time should not exceed 50.

[APP call example]

LibDevModel *devModel= [[LibDevModel alloc] init];

devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number

devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC

devModel. E eKey = @ "* *"; / / User electronic key

devModel.devType = 13; / / Device type

NSArray *cardArray = @[@” 1234562210”, @” 1548764250”, @” 1475478421”];

int ret = [LibDevModel writeCardToDevice: devModel andCards: cardArray];

[LibDevModel onControlOver:^(int ret, NSMutableDictionary *msgDict) {

```
        if (ret == SUCCESS)
    {
    / / Configuration was successful
    }
    else
    {
    / / Configuration failure View Supplementary Table III as per the ret return value
    }
    }];

if(ret != 0)
  {
      / / Configuration failed, view attached Table III from the ret return value
  }
```

4.18 deleteCardFromDevice

[function]

Delete the card operation interface, and delete the card number from the device in batch.

The deleteCardFromDevice function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)deleteCardFromDevice:(LibDevModel *)devModel andCards:(NSArray *)cardArray;
```

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

cardArray: Card number data, card number is decimal within ten, the number of card numbers written each time can not exceed 50.

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
NSArray *cardArray = @[@”1234562210”, @”1548764250”, @”1475478421”];
int ret = [LibDevModel deleteCardFromDevice: devModel andCards: cardArray ];
[LibDevModel onControlOver:^(int ret, NSMutableDictionary *msgDict) {
    if (ret == SUCCESS)
{
/ / Deleted successfully
}
else
```

```
    {
    / / Delete failure to view Schedule III according to the ret return value
    }
    }];

if(ret != 0)
  {
      / / Delete failed, view Schedule III based on ret return value
  }
```

4.19 getCardNumbersFromDevice

[function]

Read the card interface to obtain the card number data stored in the device from the device.

The getCardNumbersFromDevice function is the static method of the Lib DevModel class. The return values are detailed in Schedule III.

[function]

+(int)getCardNumbersFromDevice:(LibDevModel *)devModel andProgress:(BlockOnReadCardProgressOver)progress andComplete:(BlockOnReadCardCompleteOver)complete;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

progress: Read the callback of the card process to display the progress.

complete: Read the callback when the card completes or any other exception.

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
int ret = [LibDevModel getCardNumbersFromDevice:devModel andProgress:^(int cur, int all){
    / / The cur represents the number of card numbers currently acquired, and the all represents the total number of card numbers in the device
} andComplete:^(int result, int all, NSArray<NSString *>*cardnumbers){
    / / result represents the results, see Schedule 3, all for the total number of cardnumbers and cardnumbers for the card number data
}];

if(ret != 0)
```

{

/ / Failed to read the card number, and view Schedule III according to the ret return value

}

4.20 syncFingerPrintToDevice

[function]

Synchronize the fingerprint interface, the fingerprint data synchronization to the device, so as to realize the fingerprint unlock.

The syncFingerPrintToDevice function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)syncFingerPrintToDevice:(LibDevModel *)devModel andFingerprints:(NSMutableArray<DMFingerprintModel *>*)fingerprintModels andProgress:(BlockOnSyncFingerPrintProgressOver)progress andComplete:(BlockOnSyncFingerPrintCompleteOver)complete;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

fingerprintModels: Fingerprint data

progress: Synchronize the callback of the fingerprint data process, used to display the progress.

complete: Synchronize the callback when the fingerprint completes or any other abnormal situation.

[APP call example]

LibDevModel *devModel= [[LibDevModel alloc] init];

devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number

devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC

devModel. E eKey = @ "* *"; / / User electronic key

devModel.devType = 13; / / Device type

DMFingerprintModel * model = [[DMFingerprintModel alloc] init]; / / Create the fingerprint data

model.identity = @ "123456"; / / user identity id, whose size is not more than 4 bytes

NSString *base64Str = @”154sd34af5f43d215....."; / / The print data represented by base64 character

NSData * data = [NSData dataWithBase64EncodeString: base64Str]; / / base64 converted to NSData

model.fingerPrintDatas = @[data];

NSArray *fingerprints = @[model];

int ret = [LibDevModel syncFingerPrintToDevice:devModel andFingerprints: fingerprints andProgress:^(int cur, int all){

/ / The cur represents the number of fingerprints currently synchronized, and the all represents the total number of fingerprints required to be synchronized

} andComplete:^(int result, int all){

/ / result represents the results, see Table III, all for the total number of fingerprints

}];

if(ret != 0)

{

/ / Synchronize fingerprint failed, view Schedule III according to the ret return value

}

4.21 getFingerPrintsFromDevice

[function]

Read the fingerprint data interface and read out all the fingerprint data in the device.

The getFingerPrintsFromDevice function is the static method of the Lib DevModel class. The return values are detailed in Schedule III.

[function]

+(int)getFingerPrintsFromDevice:(LibDevModel *)devModel andProgress:(BlockonReadFingerPrintProgressOver)progress andComplete:(BlockonReadFingerPrintComplete)complete;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

progress: Read the callback of the fingerprint data process to display the progress.

complete: Read the callback when the fingerprint is complete or other exceptions.

[APP call example]

LibDevModel *devModel= [[LibDevModel alloc] init];

devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number

devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC

devModel. E eKey = @ "* *"; / / User electronic key

devModel.devType = 13; / / Device type

int ret = [LibDevModel getFingerPrintsFromDevice:devModel andProgress:^(int curFP, int allFP, int curPackage, int allPackage){

/ / The curFP represents the number of fingerprints currently read, the allFP represents the total number of fingerprints in the device, intPackage represents the number of currently synchronized fingerprints and allPackage represents the total number of currently synchronized fingerprints

} andComplete:^(int result, int all, NSArray *fingerprints){

/ / result for the results, see Schedule 3, all for the total number of fingerprints, fingerprint data (DMFingerprintModel array)

}];

if(ret != 0)

{

/ / Failed to obtain the fingerprint, and view Schedule III according to the ret return value

}

4.22 getOpenDoorRecordFromDevice

[function]

Read the door opening recording interface of the device, read all the door opening records in the device, including Bluetooth opening, credit card opening, touch opening, password opening, fingerprint opening, remote opening, etc.

The getOpenDoorRecordFromDevice function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)getOpenDoorRecordFromDevice:(LibDevModel *)devModel andProgress:(BlockonReadOpenDoorRecordProgressOver)progress andComplete:(BlockonReadOpenDoorRecordCompleteOver)complete;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

progress: Read the callback of the door opening data process to display the progress.

complete: Read the callback when the door opening record is completed or other abnormal situation.

[APP call example]

LibDevModel *devModel= [[LibDevModel alloc] init];

devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number

devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC

devModel. E eKey = @ "* *"; / / User electronic key

devModel.devType = 13; / / Device type

int ret = [LibDevModel getOpenDoorRecordFromDevice:devModel andProgress:^(int cur, int all){

/ / The cur represents the number of door records currently read, and all represents the number of all doors records

} andComplete:^(int result, int all, NSArray *records){

/ * result represents the result, see Schedule 3, all represents the total number of open door records, records: Open door record data, data composed of NSDictionary, including the following fields:

serialNumber:, Serial number, NSString type

Date: Date, NSString type yyyyMMddHHmmss format

Type: Type, NSString type, can be converted to int type, 1: Bluetooth open, 2: swipe card open, 3: password open, 4: remote open, 5: touch open, 6: fingerprint open

result: Result NSString type, 0: success, 1: failure

cardno: Card number, NSString type (applicable: card opening, Bluetooth opening device)

userIdentity: User id, NSString type (applicable: Bluetooth door, fingerprint door)

Password: Password, NSString type (applicable: password open door)*/

}];

if(ret != 0)

{

/ / Failed to obtain the door opening record, and view Schedule III according to the ret return value

}

4.23 getRecentOpenDoorRecordFromDevice

[function]

Read the door opening recording interface of the device, read all the door opening records in the device, including Bluetooth opening, credit card opening, touch opening, password opening, fingerprint opening, remote opening, etc.

The getRecentOpenDoorRecordFromDevice function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)getRecentOpenDoorRecordFromDevice:(LibDevModel *)devModel readCount:(int)readCount andProgress:(BlockonReadOpenDoorRecordProgressOver)progress andComplete:(BlockonReadOpenDoorRecordCompleteOver)complete;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

readCount: Call the maximum number of interface reads (read at most 15 records), that is, if readCount is 5, the interface can read at most 5 times, that is, return at most 75 records. If you want to directly read all the unread records, readCount can be set to 0

progress: Read the callback of the door opening data process to display the progress.

complete: Read the callback when the door opening record is completed or other abnormal situation.

[APP call example]

See 4.20, the returned open door data format is universal

4.24 cleanAllOpenDoorRecords

[function]

Clear the door opening records stored in the device.

The cleanAllOpenDoorRecords function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)cleanAllOpenDoorRecords:(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

callBack: Clear the callback of the door opening door record

4.25 setDeviceStaticIP

[function]

Set the static IP interface and the static IP of the equipment, suitable for V500.

The SetDeviceStaticIP function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)setDeviceStaticIP:(LibDevModel *)devModel andEnableStaticIP:(BOOL)enable andStaticIP:(NSString *)staticIP andMask:(NSString *)mask andGate:(NSString *)gate andDns:(NSString *)dns andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

enable: YES: With the staticIP, the other parameters (staticIP, mark, gate, dns) cannot be empty. NO: using DHCP, then the other parameters (staticIP, mask, gate, dns) can be directly imported into the nil.

staticIP: Static IP

Mask: the subnet mask

gate: gateway

The dns: the DNS server

complete: Set the callback for complete or other exceptions.

[APP call example]

LibDevModel *devModel= [[LibDevModel alloc] init];

devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number

devModel.devMac = @ "* * * * * * * * * * * * *"; / / Device MAC

devModel. E eKey = @ "* *"; / / User electronic key

devModel.devType = 1; / / Device type

NSString *staticIP = @”192.168.1.251”;

NSString *mask = @”255.255.255.0”;

NSString *gate = @”192.168.1.1”;

NSString *dns = @”8.8.8.8”;

```
int ret = [LibDevModel setDeviceStaticIP:devModel andEnableStaticIP:YES andStaticIP:staticIP andMask:mask andGate:gate andDns:dns andCallback:^(int ret, NSMutableDictionary *msgDict) {
    if (ret == 0)
{
/ / Configuration was successful
}
else
{
/ / Configuration failure View Supplementary Table III as per the ret return value
}
}];

    if(ret != 0)
    {
      / / Configuration failure View Supplementary Table III as per the ret return value
    }
```

4.26 setServerIP

[function]

Set the server address of the device connection, suitable for V500 and other devices.

The SetDeviceStaticIP function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)setServerIP:(LibDevModel *)devModel andServerIP:(NSString *)serverIP andServerPort:(int)serverPort andCallback:(BlockOnControlOver)callBack;
```

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

serverIP: Connect to the IP of the server.

serverPort: Connect the port to the server

complete: Set the callback for complete or other exceptions.

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 1; / / Device type
NSString *serverIP= @” 192.168.1.251”;
int serverPort= 8089;
int ret = [LibDevModel setServerIP:devModel andServerIP:serverIP
```

```
andServerPort:serverPort    andCallback:^(int ret, NSMutableDictionary *msgDict) {
      if (ret == 0)
{
/ / Configuration was successful
}
else
{
/ / Configuration failure View Supplementary Table III as per the ret return value
}
}];

      if(ret != 0)
      {
         / / Configuration failure View Supplementary Table III as per the ret return value
      }
```

4.27 getSwipeAddCardno

[function]

Obtain the card number added by the device swipe card addition mode, which is suitable for door lock devices.

The getSwipeAddCardno function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)getSwipeAddCardno:(LibDevModel  *)devModel  andCallBack:(Block0
nControl0ver)callBack ;
```

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

callBack: A callback

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic
key
devModel.devType = 1; / / Device type
int ret = [LibDevModel getSwipeAddCardno :devModel andCallback:^(int ret,
NSMutableDictionary *msgDict) {
if (ret == 0)
{
/ / Success
```

```
        / / msgDict [@ "cardno"] is the card number (NSString *)
        / / msgDict [@ "cardType"] is card type (NSNumber *) 1 is IC card, 2 is ID card
}
else
{
/ / Obtain failure to view Schedule III according to the ret return value
}
}];

    if(ret != 0)
    {
      / / Obtain failure to view Schedule III according to the ret return value
    }
```

4.28 getDeviceSignal

[function]

Obtain the signal value of the device.

The getDeviceSignal function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)getDeviceSignal:(LibDevModel  *)devModel  andCallback:BlockOnControlOver)callBack;
```

[parameter declaration]

devModel: LibDevModel device object, please refer to Table I Class LibDevModel for details.

callBack : callback。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic
key
devModel.devType = 13; / / Device type
int ret = [LibDevModel getDeviceSignal: devModel andCallback:^(int ret,
NSMutableDictionary *msgDict) {
    if (ret == 0)
{
/ / Success
    / / msgDict [@ "signal"] is a signal (NSNumber *)
}
else
```

```
{
/ / Obtain failure to view Schedule III according to the ret return value
}
}];

if(ret != 0)
   {
        / / Failed to obtain, view Schedule III based on the ret return value
   }
```

4.29 scanAndOpenDoor

[function]

One key to open the door.

The scanAndOpenDoor function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)scanAndOpenDoor:(NSArray<LibDevModel *> *)devList timeout:(int)timeout callback:(BlockOnControlOver)callBack;
```

[parameter declaration]

devList: Device list.

timeout: Scan timeout time, recommended at 5000. Unit in milliseconds

callBack : callback。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
NSMutableArray *devices = [NSMutableArray    array];
[devices addObject:devModel];
int ret = [LibDevModel scanAndOpenDoor: devices timeout:5000 callback:^(int ret, NSMutableDictionary *msgDict) {
        if (ret == 0)
{
/ / Open the door successfully
        / / msgDict [@ "devSn"] is the device serial number that was operated successfully
}
else
{
/ / Failed to view Schedule III according to the ret return value
```

```
        / / msgDict [@ "devSn"] is the serial number of the device that failed to open the door, this is not necessarily there, because it may not scan the device
}
}];

if(ret != 0)
   {
        / / Failed, view Schedule III from the ret return value
   }
```

4.30 scanAndOpenDoor:process

[function]

One key to open the door, with the middle process callback.

The scanAndOpenDoor function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)scanAndOpenDoor:(NSArray<LibDevModel *> *)devList timeout:(int)timeout process:(BlockOnControlOver)process callback:(BlockOnControlOver)callBack;
```

[parameter declaration]

devList: Device list.

timeout: Scan timeout time, recommended at 5000. Unit in milliseconds

process: Intermediate process callback

callBack : callback。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
NSMutableArray *devices = [NSMutableArray    array];
[devices addObject:devModel];
int ret = [LibDevModel scanAndOpenDoor: devices timeout:5000 process:^(int ret, NSMutableDictionary *msgDict) {
     if (ret == -121)
{
    / / In scanned
}
else    if(ret == -122)
```

```
{
        / / Opening the door msgDict [@ "devSn"] is the serial number of the device opening
    the door
    } }   callback:^(int ret, NSMutableDictionary *msgDict) {
      if (ret == 0)
{
/ / Open the door successfully
        / / msgDict [@ "devSn"] is the device serial number that was operated successfully
}
else
{
/ / Failed to view Schedule III according to the ret return value
        / / msgDict [@ "devSn"] is the serial number of the device that failed to open the door,
this is not necessarily there, because it may not scan the device
}
}];

if(ret != 0)
  {
        / / Failed, view Schedule III from the ret return value
  }
```

4.31 deviceRegistFingerprint

[function]

Device registration fingerprint.

The deviceRegistFingerprint function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)deviceRegistFingerprint:(LibDevModel *)device fingerprintID:(int)fingerprintID andCallbac
k:(BlockOnControlOver)callBack;
```

[parameter declaration]

device: Equipment.

fingerprintID: fingerprintID, int type, 4 bytes

callBack: callback, note that this callback will be called 1~2 times. If the return value of the first callback is 56, it will be called back once later.

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic
```

```
key
      devModel.devType = 13; / / Device type
           int ret = [LibDevModel deviceRegistFingerprint: devModel fingerprintID:10
      callback:^(int ret, NSMutableDictionary *msgDict) {
        if (ret == 56)
{
/ / The fingerprint is being registered
}
      else if(ret == 0){
           / / Successful registration
      }
else
{
/ / Failed to view Schedule III according to the ret return value
}
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.32 getFingerprintIDsFronDevice

[function]

Get all the fingerprint ID in the device.

The getFingerprintIDsFromDevice function is the static method of the Lib DevModel class. The return values are detailed in Schedule III.

[function]

```
+(int)getFingerprintIDsFromDevice:(LibDevModel *)device andCallback:(BlockOnControlOver)ca
llBack;
```

[parameter declaration]

device: Equipment.

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic
```

key

```
devModel.devType = 13; / / Device type
    int ret = [LibDevModel getFingerprintIDsFromDevice: devModel callback:^(int ret,
NSMutableDictionary *msgDict) {
    if (ret == 0)
{
// success
            msgDict [@ "fingerprintIDs"] is the list of fingerprint ids
}
else
{
/ / Failed to view Schedule III according to the ret return value
}
}];

if(ret != 0)
  {
    / / Failed, view Schedule III from the ret return value
  }
```

4.33 setAPN

[function]

Set up the device APN.

The setAPN function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)setAPN:(LibDevModel *)devModel andApnName:(NSString *)apnName andUserName:(NSString *)userName andPwd:(NSString *)pwd andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

apnName: apn name

userName: Apn user name

pwd: The apn user password

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic
```

```
key
    devModel.devType = 13; / / Device type
        int ret = [LibDevModel setAPN: devModel andApnName:@ ” cmiot ”
    andUserName:@”” andPwd:@”” andCallback:^(int ret, NSMutableDictionary *msgDict)
    {
            if (ret == 0)
    {
    // success
    }
    else
    {
    / / Failed to view Schedule III according to the ret return value
    }
    }];

if(ret != 0)
  {
     / / Failed, view Schedule III from the ret return value
  }
```

4.34 getAPN

[function]

Get the device APN.

The getAPN function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)getAPN:(LibDevModel *)devModel callback:(BlockOnControlOver)callBack;
```

[parameter declaration]

devModel: Equipment.

apnName: apn name

userName: Apn user name

pwd: The apn user password

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic
```

key

devModel.devType = 13; / / Device type

int ret = [LibDevModel getAPN: devModel callback:^(int ret, NSMutableDictionary *msgDict) {

if (ret == 0)

{

/ / Success msgDict Dictionary of apnName: apn name, apnUserName: apn user name, apnPwd: apn password

}

else

{

/ / Failed to view Schedule III according to the ret return value

}

}];

```
if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.35 setNBNetworkSetting

[function]

Set the NB networking parameters.

The SetNBNetworkSetting function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)setNBNetworkSetting:(LibDevModel *)devModel connectCount:(int)connectCount heartBeatInterval:(int)heartBeatInterval synFlag:(int)synFlag synInterval:(int)synInterval andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

connectCount: The number of networking times 1~5 is not set to-1

heartBeatInterval: Heart rate interval 1~240 is not set to-1

synFlag : The synchronous data flag 0,1 is not set to-1

synInterval: The time interval between 1 and 240 is not set to-1

callBack : callback 。

[APP call example]

LibDevModel *devModel= [[LibDevModel alloc] init];

devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number

```
devModel.devMac = @ "* * * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic
key
devModel.devType = 13; / / Device type
    int ret = [LibDevModel setNBNetworkSetting: devModel connectCount:3
heartBeatInterval:100 synFlag:1 synInterval:200 andC allback:^(int ret, NSMutableDictionary
*msgDict) {
        if (ret == 0)
{
// success
}
else
{
/ / Failed to view Schedule III according to the ret return value
}
}];

if(ret != 0)
  {
    / / Failed, view Schedule III from the ret return value
  }
```

4.36 setNBNetworkConnectionPwd

[function]

Set the NB networking password.

The setNBNetworkConnectionPwd function is the static method of the Lib DevModel class. The return values are detailed in Schedule III.

[function]

```
+(int)setNBNetworkConnectionPwd:(LibDevModel *)devModel oldPassword:(NSString *)oldPas
sword newPassword:(NSString *)newPassword callback:(BlockOnControlOver)callBack;
```

[parameter declaration]

devModel: Equipment.

oldPassword: The old password is a pure number, within 4 bytes

newPassword: The new password is a pure number, within 4 bytes

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
```

devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* *"; / / User electronic key
devModel.devType = 13; / / Device type
int ret = [LibDevModel setNBNetworkConnectionPwd: devModel oldPassword:@ ” 123456” newPassword:@ ” 666666” callback:^(int ret, NSMutableDictionary *msgDict) {
if (ret == 0)
{
// success
}
else
{
/ / Failed to view Schedule III according to the ret return value
}
}];

```
if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.37 getSimstatus

[function]
Get the sim card information.
The setNBNetworkConnectionPwd function is the static method of the Lib DevModel class.
The return values are detailed in Schedule III.

[function]
+(int)getSimstatus:(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;
[parameter declaration]
devModel: Equipment.
callBack : callback 。
[APP call example]
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* *"; / / User electronic key
devModel.devType = 13; / / Device type

```
        int ret = [LibDevModel getSimstatus: devModel andCallback:^(int ret,
    NSMutableDictionary *msgDict) {
                if (ret == 0)
    {
    // success
                msgDict Dictionary key description:
                simModelStatus Module state
                simCardStatus Card status
                simModelSn Module serial number
                simICCID ICCID
                simLACID Base Station ID
                simCommunityID Community ID
                simModelName Module name
    }
    else
    {
    / / Failed to view Schedule III according to the ret return value
    }
    }];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.38 deviceUpgradeByNetwork

[function]

Upgrade the device.(Arrange the device network in advance before using this function)

The deviceUpgradeByNetwork function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)deviceUpgradeByNetwork:(LibDevModel *)devModel url:(NSString *)url andCallback:(Blo
ckOnControlOver)callBack;
```

[parameter declaration]

devModel: Equipment.

The url: the address of the upgrade package

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
```

devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* *"; / / User electronic key
devModel.devType = 13; / / Device type
int ret = [LibDevModel deviceUpgradeByNetwork: devModel url:@” http://*****”
andCallback:^(int ret, NSMutableDictionary *msgDict) {
if (ret == 0)
{
/ / The command was sent successfully and the device is being upgraded
}
else
{
/ / Failed to view Schedule III according to the ret return value
}
}];

```
if(ret != 0)
  {
     / / Failed, view Schedule III from the ret return value
  }
```

4.39 cleanFrequencyban

[function]

Clear frequency (nb module), only for NB devices

The cleanFrequencyban function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)cleanFrequencyban :(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

callBack : callback 。

[APP call example]

LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* *"; / / User electronic key
devModel.devType = 13; / / Device type

```
        int ret = [LibDevModel cleanFrequencyban : devModel     andCallback:^(int ret,
    NSMutableDictionary *msgDict) {
                if (ret == 0)
                    {
              // success
                    }
                    else
                    {
                    / / Failed to view Schedule III according to the ret return value
                    }
    }];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.40 getNBParamInfo

[function]

Read the respective parameter values of the NB base station

The getNBParamInfo function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)getNBParamInfo :(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;
```

[parameter declaration]

devModel: Equipment.

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic
key
devModel.devType = 13; / / Device type
        int ret = [LibDevModel getNBParamInfo : devModel     andCallback:^(int ret,
    NSMutableDictionary *msgDict) {
                if (ret == 0)
                    {
              // success
                        msgDict Dictionary key description:
```

```
                    earfcn          int
                    earfcnOffset    int
                    pci             int
                    rsrp            int
                    rsrq            int
                    rssi            int
                    snr             int
                    band            int
                    ecl             int
                    txPwr           int
                    lacID           int
                    cid             int
                }
                else
                {
                / / Failed to view Schedule III according to the ret return value
                }
    }];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.41 setCPUCardKey

[function]

Set up the CPU card key

The setCPUCardKey function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)setCPUCardKey :(LibDevModel *)devModel key:(NSString *)key andCallback:(BlockOnControlOver)callBack;
```

[parameter declaration]

devModel: Equipment.

Key: 32-bit cpu key (i. e., 16 bytes)

callBack : callback 。

[APP call example]

LibDevModel *devModel= [[LibDevModel alloc] init];

devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number

devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC

devModel. E eKey = @ "* *"; / / User electronic key

devModel.devType = 13; / / Device type

int ret = [LibDevModel setCPUCardKey : devModel key:@ ” 11223344556677889900112233445566 ” andCallback:^(int ret, NSMutableDictionary *msgDict) {

if (ret == 0)

{

// success

}

else

{

/ / Failed to view Schedule III according to the ret return value

}

}];

```
if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.42 setNBUnicomPlatformParams

[function]

Set the nb Unicom platform parameters

The setNBUnicomPlatformParams function is the static method of the Lib DevModel class. The return values are detailed in Schedule III.

[function]

+(int)setNBUnicomPlatformParams :(LibDevModel *)devModel unlinkPk:(NSString *)unlinkPk unlinkDevSecret:(NSString *)unlinkDevSecret andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

unlinkPk: Parameters of Unicom platform, no more than 40 characters (ASCII)

unlinkDevSecret: Parameters of Unicom platform, no more than 40 characters (ASCII)

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
    int ret = [LibDevModel setNBUnicomPlatformParams : devModel unlinkPk :@ ” 12349437 ” unlinkDevSecret:@ ” 42343249532423 ” andCallback:^(int ret, NSMutableDictionary *msgDict) {
        if (ret == 0)
        {
    // success
        }
        else
        {
        / / Failed to view Schedule III according to the ret return value
        }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.43 getNBUnicomPlatformParams

[function]

Set the nb Unicom platform parameters

The getNBUnicomPlatformParams function is the static method of the Lib DevModel class. The return values are detailed in Schedule III.

[function]

```
+(int)getNBUnicomPlatformParams :(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;
```

[parameter declaration]

devModel: Equipment.

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic
key
devModel.devType = 13; / / Device type
      int ret = [LibDevModel getNBUnicomPlatformParams : devModel    andCallback:^(int
ret, NSMutableDictionary *msgDict) {
            if (ret == 0)
            {
      // success
                  msgDict Dictionary key description:
                  unlinkPk:     NSString
                  unlinkDevSecret:    NSString
            }
            else
            {
            / / Failed to view Schedule III according to the ret return value
            }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.44 setCardTypeVerifyType

[function]

Set the read card type and the card verification type

The setCardTypeVerifyType function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)setCardTypeVerifyType :(LibDevModel *)devModel cardType:(int)cardType verifyType:(int)
```

verifyType andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

cardType: Read card type 1: A card 2: B card 3: AB card

verifyType: Card verification type 1: Card number only 2: Card number or sector key 3: Card number plus sector key 4: CPU card

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
      int ret = [LibDevModel setCardTypeVerifyType : devModel    cardType:1 verifyType:2 andCallback:^(int ret, NSMutableDictionary *msgDict) {
                  if (ret == 0)
            {
        // success
            }
            else
            {
            / / Failed to view Schedule III according to the ret return value
            }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.45 getCardTypeVerifyType

[function]

Read the read card type and the card verification type

The getCardTypeVerifyType function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)getCardTypeVerifyType :(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
    int ret = [LibDevModel getCardTypeVerifyType : devModel andCallback:^(int ret, NSMutableDictionary *msgDict) {
                if (ret == 0)
        {
    // success
                msgDict Dictionary key description:
                cardType    int
                verifyType    int
        }
        else
        {
        / / Failed to view Schedule III according to the ret return value
        }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.46 enableAntiCopy

[function]

Open the anti-replication function not

The enableAntiCopy function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)enableAntiCopy :(LibDevModel *)devModel enable:(BOOL)enable andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

enable: Open or not

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
      int ret = [LibDevModel enableAntiCopy : devModel    enable:YES andCallback:^(int ret, NSMutableDictionary *msgDict) {
                  if (ret == 0)
            {
      // success
            }
            else
            {
            / / Failed to view Schedule III according to the ret return value
            }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.47 setWiganOutputparams

[function]

Set the Weigen output parameters

The setwiganOutputparams function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)setwiganOutputparams :(LibDevModel *)devModel isReal:(BOOL)isReal isSequence:(BOOL)isSequence virtualCard:(NSString *)card andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

isReal: Whether to use the real card YES, the real card NO virtual card

isSequence: YES in straight order NO in inverse order

Card: If it is a virtual card mode, then output the card number

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
      int ret = [LibDevModel setwiganOutputparams : devModel    isReal:YES isSequence:YES virtualCard:nil andCallback:^(int ret, NSMutableDictionary *msgDict) {
                  if (ret == 0)
            {
      // success
            }
            else
            {
            / / Failed to view Schedule III according to the ret return value
            }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.48 getWiganOutputparams

[function]

Set the Weigen output parameters

The getwiganOutputparams function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)getwiganOutputparams :(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
    int ret = [LibDevModel getwiganOutputparams : devModel andCallback:^(int ret, NSMutableDictionary *msgDict) {
                if (ret == 0)
            {
        // success
                msgDict Dictionary key description:
                cardNumberMode          int
                wiganOutputByteOrder    int
                virtualCardNumber NSString (Virtual card number is available only when the cardNumberMode is 1)
            }
            else
            {
            / / Failed to view Schedule III according to the ret return value
            }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.49 setNormallyOpencloseMode

[function]

Set the Weigen output parameters

The setNormallyOpencloseMode function is the static method of the Lib DevModel class. The return values are detailed in Schedule III.

[function]

+(int)setNormallyOpencloseMode :(LibDevModel *)devModel isNormallyOpen:(BOOL)normalOpen andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

normalOpen: Whether often open

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
      int ret = [LibDevModel setNormallyOpencloseMode : devModel   isNormallyOpen：NO
andCallback:^(int ret, NSMutableDictionary *msgDict) {
                  if (ret == 0)
            {
        // success
            }
            else
            {
            / / Failed to view Schedule III according to the ret return value
            }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.50 getNormallyOpencloseMode

[function]

Set the Weigen output parameters

The getNormallyOpencloseMode function is the static method of the Lib DevModel class. The return values are detailed in Schedule III.

[function]

+(int)getNormallyOpencloseMode :(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
    int ret = [LibDevModel getNormallyOpencloseMode : devModel    andCallback:^(int ret, NSMutableDictionary *msgDict) {
                    if (ret == 0)
            {
        // success
                    msgDict Dictionary key description:
                    normalOpenclosedMode          int
            }
            else
            {
            / / Failed to view Schedule III according to the ret return value
            }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.51 getCpuCardkey

[function]

Set the Weigen output parameters

The getCpuCardkey function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)getCpuCardkey :(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
    int ret = [LibDevModel getCpuCardkey : devModel andCallback:^(int ret, NSMutableDictionary *msgDict) {
                if (ret == 0)
        {
    // success
                msgDict Dictionary key description:
                cpuKey      NSString
        }
        else
        {
        / / Failed to view Schedule III according to the ret return value
        }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.52 configHostWifi

[function]

Set up the host for WiFi

The configHostWifi function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)configHostWifi :(LibDevModel *)devModel WiFiName:(NSString *)WiFiName WiFiPwd:(NSString *)WiFiPwd andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

WiFiName: Wifi name, it cannot be empty

WiFiPwd: Wifi password, it can be empty

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
      int ret = [LibDevModel configHostWifi : devModel    WiFiName:@”wifi ” WiFiPwd:@” 12345678”  andCallback:^(int ret, NSMutableDictionary *msgDict) {
                  if (ret == 0)
            {
                  // success
            }
            else
            {
            / / Failed to view Schedule III according to the ret return value
            }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.53 upgradeEnable

[function]

Upgrade enabled for RD02 Series Bluetooth devices (such as 618Pro-v3, VF711C, etc.)

The upgradeEnable function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)upgradeEnable :(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
    int ret = [LibDevModel upgradeEnable : devModel andCallback:^(int ret, NSMutableDictionary *msgDict) {
                if (ret == 0)
        {
                // success
        }
        else
        {
        / / Failed to view Schedule III according to the ret return value
        }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.54 getHostNetworkInfo

[function]

Obtain the host device network status information

The getHostNetworkInfo function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)getHostNetworkInfo :(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBa

ck;

[parameter declaration]

devModel: Equipment.

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
    int ret = [LibDevModel getHostNetworkInfo : devModel andCallback:^(int ret, NSMutableDictionary *msgDict) {
                if (ret == 0)
            {
                // success
                msgDict Dictionary key description:
                cardNum NSString 4g Card number
                commType NSNumber Communication type 0: MQTT 1: HTTP 2: TCP
                connectStatus NSNumber Connect status
                i ccid NSString sim Card id
                networkType NSNumber Network type 0: no network 1: Ethernet 2: WiFi 3:4G
                rssi NSNumber Signal value
                serverOnline NSNumber Whether the cloud is online 0: offline 1: online
                voipOnline NSNumber intercom online 0: Offline 1: online
            }
            else
            {
            / / Failed to view Schedule III according to the ret return value
            }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.55 getHostDeviceInfo

[function]

Obtain the host device information

The getHostDeviceInfo function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)getHostDeviceInfo :(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
    int ret = [LibDevModel getHostNetworkInfo : devModel andCallback:^(int ret, NSMutableDictionary *msgDict) {
                if (ret == 0)
            {
                // success
                msgDict Dictionary key description:
                cardCount NSNumber Number of card numbers
                dhcpMode NSNumber Device ip mode, 0: DHCP 1: static IP
                faceCount NSNumber Number of human faces
                ip NSString Device ip
                serverAddr NSString, Device cloud server address
                sipAddr NSString Device intercom server address
                time NSString Device time
                userCount NSNumber Number of users
                v ersion NSString Device version
                wifiName NSString WiFi Name
                wifiPwd NSString WiFi Password
            }
            else
            {
            / / Failed to view Schedule III according to the ret return value
            }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
```

}

4.56 setHostAPN

[function]

Set up the host apn

The setHostAPN function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)setHostAPN :(LibDevModel *)devModel apnName:(NSString *)apnName userName:(NSS tring *)userName pwd:(NSString *)pwd mnc:(NSString *)mnc mcc:(NSString *)mcc andCallb ack:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

apnName: apn name

userName: User name

pwd: password

mnc: mnc

mcc: mcc

callBack : callback 。

[APP call example]

LibDevModel *devModel= [[LibDevModel alloc] init];

devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number

devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC

devModel. E eKey = @ "* *"; / / User electronic key

devModel.devType = 13; / / Device type

```
int ret = [LibDevModel setHostAPN : devModel apnName:@ ” cmiot ” userName:@ ” ” pwd:@ ” ” mnc:@ ” 08 ” mcc:@ ” 460 ” andCallback:^(int ret, NSMutableDictionary *msgDict) {
        if (ret == 0)
    {
        // success
    }
    else
    {
    / / Failed to view Schedule III according to the ret return value
    }
}];

if(ret != 0)
```

```
{
    / / Failed, view Schedule III from the ret return value
}
```

4.57 getHostAPNInfo

[function]

Get the host APN information

The getHostAPNInfo function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)getHostAPNInfo :(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;
```

[parameter declaration]

devModel: Equipment.

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic
key
devModel.devType = 13; / / Device type
    int ret = [LibDevModel getHostAPNInfo : devModel andCallback:^(int ret,
NSMutableDictionary *msgDict) {
                if (ret == 0)
        {
                // success
                msgDict Dictionary key description:
                apnName NSString apn Name
                userName NSString User name
                pwd NSString Password
                mcc NSString mcc
                mnc NSString mnc
        }
        else
        {
        / / Failed to view Schedule III according to the ret return value
        }
}];
```

```
if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.58 setHostCommType

[function]

Get the host APN information

The setHostCommType function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

```
+(int)setHostCommType:(LibDevModel *)devModel commType:(int)commType andCallback:(BlockOnControlOver)callBack;
```

[parameter declaration]

devModel: Equipment.

commType: Communication type 0: MQTT 1: HTTP 2: TCP

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
      int ret = [LibDevModel setHostCommType : devModel    commType:0 andCallback:^(int ret, NSMutableDictionary *msgDict) {
                  if (ret == 0)
            {
                  // success
            }
            else
            {
            / / Failed to view Schedule III according to the ret return value
            }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.59 setNormallyOpenEnable

[function]

Set whether the host is often open open

The setNormallyOpenEnable function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)setNormallyOpenEnable :(LibDevModel *)devModel enable:(int)enable andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

enable: 0 Off 1 is turned on

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
    int ret = [LibDevModel setNormallyOpenEnable : devModel enable:1 andCallback:^(int ret, NSMutableDictionary *msgDict) {
                if (ret == 0)
            {
                // success
            }
            else
            {
            / / Failed to view Schedule III according to the ret return value
            }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

4.60 getNormallyOpenEnable

[function]

Read whether the host is often open or not

The getNormallyOpenEnable function is the static method of the Lib DevModel class.

The return values are detailed in Schedule III.

[function]

+(int)getNormallyOpenEnable :(LibDevModel *)devModel andCallback:(BlockOnControlOver)callBack;

[parameter declaration]

devModel: Equipment.

callBack : callback 。

[APP call example]

```
LibDevModel *devModel= [[LibDevModel alloc] init];
devModel.devSn = @ "* * * * * * * * * *"; / / Device serial number
devModel.devMac = @ "* * * * * * * * * * * *"; / / Device MAC
devModel. E eKey = @ "* * * * * * * * * * * * * * * * * * * * * * * * * * * * *"; / / User electronic key
devModel.devType = 13; / / Device type
    int ret = [LibDevModel getNormallyOpenEnable : devModel andCallback:^(int ret, NSMutableDictionary *msgDict) {
                if (ret == 0)
        {
                // success
                msgDict Dictionary key description:
                normallyOpenEnable NSNumber Turn on normally 1: Turn on 0: off
        }
        else
        {
        / / Failed to view Schedule III according to the ret return value
        }
}];

if(ret != 0)
  {
      / / Failed, view Schedule III from the ret return value
  }
```

5. Appendix

Schedule 1

Class	Lib DevModel		
Attribute	**Column**	**Type**	**Remark**
	devSn	NSString	Device serial number, a must
	devMac	NSString	Device MAC, a must
	devType	int	Device type (1 access, 2 entrance control all-in-one, 3 ladder control, 4 wireless network lock, 5 bluetooth remote control module, 6 access controller, 7 touch open and close, 8 qr code all-in-one, 9 qr code reading, 10 all-in-one (write card), 11 touch open and close door (WiFi), 12 entrance guard all-in-one (WiFi, write card), 13 access control all-in-one (WiFi)), must
	privilege	int	Manager authority (1 super administrator, 2 administrator, 4 ordinary users), all-in-one machine use
	open Mode	int	Open door mode (1 mobile phone, 2 mobile phone + card, 3 mobile phone + password), default mobile phone open door
	verified	int	Verification method (1 validity period, 2 times, 3 validity period + times), default validity period
	startDate	NSString	Effective start time, format: year, month, day, time and second, such as: 20151001121000
	endDate	NSString	Freezing time, format: year, month, day, time and seconds, such as: 20161001121000
	useCount	int	number of use
	eKey	NSString	User electronic key secret key, must
	cardno	NSString	For openDoor interface use, if no incoming card number in eKey, use cardno output when the device is read
	+(int) initBluetooth;		

	+(int) initBluetoothNotShowPower;
	+(void)onBluetoothStateOver:(BlockBluetoothStateOver)blockBluetoothState;
	+(int)control Device:(LibDevModel *)devModel andOperation:(int)operation;
	+(void)onControlOver:(BlockOnControlOver)blockControl;
	+(int)openDoor :(LibDevModel *)devModel;
	+(int) scanDevice:(int)scanTime andScanOpen:(BOOL)scanOpenDoor ;
	+(void)onScanOver:(BlockOnScanOver)blockScan;
	+(int)syncDeviceTime:(LibDevModel *)devModel andTime:(NSString *) time;
	+(int)modify Pwd :(LibDevModel *)devModel and OldPwd :(NSString *) oldPwd andNewPwd:(NSString *)newPwd ;
	+(int)s etDeviceParam :(LibDevModel *)devModel andWGFmt :(int)wgfmt andDriverTime:(int)driverTime andRelaySwitch:(int)relaySwitch ;
	+(int)startBackgroundModel ;
	+(int)stopBackgroundModel ;

Note Must in red font, which is the field value that must be passed in.

Attached table 2

operation	function	explain
0x00	push plate	
0x03	Add equipment	Read head and remote control module are not supported
0x04	Set the equipment time	Read head and remote control module are not supported
0x05	change password	Read head and remote control module are not supported
0x06	Enter the registration card mode	Read head and remote control module are not supported
0x07	Enter the delete card mode	Read head and remote control module are not supported
0x08	Exit the card registration mode	Read head and remote control module are not supported
0x09	Exit the card delete mode	Read head and remote control module are not supported
0x0a	Set the device configuration	Remote control module is not supported
0x0b	Get device configuration	Remote control module is not supported
0x0c	Delete all card user information	Read head and remote control module are not supported
0x0d	system initialization	Read head and remote control module

		are not supported
0x0e	Obtain the device card registration information	Read head and remote control module are not supported
0x0f	Update the device card registration information	Read head and remote control module are not supported
0x10	Delete the device card registration information	Read head and remote control module are not supported
0x11	Obtain the device user information	Read head and remote control module are not supported
0x12	Obtain the device record information	Read head and remote control module are not supported

Attached table 3

code	explain
0	success
-1	The user does not have a card number, please assign the card number first
-2	The devSn cannot be empty
-3	devMac Cannot be empty
-4	Can eKey, cannot be empty
-5	devType value (cannot be empty or value undefined)
-6	privilege Wrong value (cannot be empty or undefined when devType is all-in-one)
-7	The openMode value is not defined
-8	verified value not defined (cannot be empty or undefined when devType is all-in-one)
-9	startDate Wrong format
-10	endDate Wrong format
-11	useCount Cannot be empty (if the verified validation mode is 2 or 3)
-12	The operation value is not defined
-13	operation Other functions are not open
-14	Illegal time to open the door, that is, not to open the door within the validity period error
-15	More than the set door opening distance
-16	Weigen format error, currently supported only for 26 and 34
-17	Door opening length range incorrect, only 1-254 seconds
-18	Electrical switch parameter value error, only support 0 electric lock control, 1 electrical switch
-19	The password must be 6 digits
-20	The card number list cannot be empty
-21	Batch card operation is up to 50 card numbers each time
-22	The card number data sent is missing
-23	The incoming callback is empty

-25	The scanTime is empty
-26	Fingerprint data is empty
-27	Fingerprint model data error
-28	Fingerprint user ID error, the number of ID can not exceed 50, each ID occupies no more than 4 bytes of memory
-93	Error in incoming parameter
-101	Mobile phone Bluetooth is not on
-103	The function for the callback object does not exist
-104	Open the door failed
-105	The equipment has no response
-106	The equipment is not nearby
-107	Bluetooth is in use, please go later
-108	Error in sec scan time unit
-109	The sec scan time value can only range from 1-60
-110	The device already has a superuser, and you must initialize the device to add the device
-111	Device MAC address is in error
-118	One key to open the door, no permission device
-119	Open the door with one key, and you do not scan any device
-120	Open the door with one key, but do not match the device with permission
-121	Open the door with one key, which is scanning
-122	Open the door, open the door
0x01	A CRC calibration error
0x02	The communication command is in the wrong format
0x03	Device Management password error
0x04	ERROR_POWER
0x05	Error reading and writing of data
0x06	The user is not registered in the device
0x07	Random number detection error
0x08	Obtain a random number in error
0x09	Command length does not match
0x0a	The Add Device mode is not entered
0x0b	devKey Detection error
0x0c	The function is not supported
0x0d	Insufficient equipment capacity
0x0e	There is no card number in the equipment
0x0f	Error in getting the card number length
0x10	Error in getting the data length
0x11	No open door record
19	Host serial port communication timeout
56	Entering the fingerprint
57	Failed to register fingerprint

58	The fingerprint is full
59	Receiving fingerprint module data timeout / fingerprint module exception
60	No fingerprint collection collected timeout / timeout
61	The fingerprint already exists